

```
print("Dhwaj Jain S012 DL Expt 1")
```

```
Dhwaj Jain S012 DL Expt 1
```

```
import numpy as np
```

```
ipVecAndBiXa=np.array([-1,1,-1,1])
ipVecAndBiXb=np.array([-1,-1,1,1])
ipVecAndBiD=np.array([-1,-1,-1,1])
```

```
ipVecAndUni=np.array([[0,0,0], [0,1,0], [1,0,0], [1,1,1]] )
```

```
ipVecOrUni=np.array([[0,0,0], [0,1,1], [1,0,1], [1,1,1] ])
```

```
ipVecOrBi=np.array([-1,-1,-1], [-1,1,1], [1,-1,1], [1,1,1] ])
```

```
def UniActFn(net):
    if (net >= 0):
        fnet=1
    elif (net<0):
        fnet=0
    return fnet
```

```
def BiActFn(net):
    if (net <0):
        fnet=-1
    elif (net==0):
        fnet=0
    else:
        fnet=1
```

```
for j in range (0,4):
    e=()
```

```
c=1
for i in range (0,4):
    deltaw= c* e * (ipVecAndUni[i][0])
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-949129881.py in <cell line: 0>()
      1 c=1
      2 for i in range (0,4):
----> 3     deltaw= c* e * (ipVecAndUni[i][0])

NameError: name 'e' is not defined
```

```
w1 =0
w2=0
c=1
b=0
net(w1, w2, ipVecAndBi,b)
```

```
0
```

```
def net(w1, w2, ipVecAndBi, b):
    for i in range (0,4):
        nett= (w1* ipVecAndBiXa[i]) + (w2 * ipVecAndBiXb[i] )+ b
        q= BiActFn(nett)
    print(nett)
```

```

import numpy as np

def bipolar_step_function(net):
    """Bipolar activation function."""
    if net > 0:
        return 1
    elif net == 0:
        return 0
    else:
        return -1

def train_perceptron(inputs, targets, gate_name):
    """Trains a single-layer perceptron for a given gate using bipolar inputs and targets."""
    # Initialize weights and bias as specified
    w1, w2, b = 0.0, 0.0, 0.0
    learning_rate = 1.0 # Learning rate as specified
    epochs = 0

    print(f"--- Perceptron for {gate_name} Gate (Bipolar) ---")
    print(f"Initial weights: w1={w1}, w2={w2}, bias={b}")
    print(f"Learning Rate: {learning_rate}\n")

    # Training loop
    while True:
        epochs += 1
        print(f"Epoch {epochs}:")
        all_correct_in_epoch = True

        # Print header for the detailed table
        print(f"{'x1':<5}{ 'x2':<5}{ 'd':<5}{ 'net':<8}{ 'theta':<8}{ 'error':<8}{ 'dw1':<8}{ 'dw2':<8}{ 'db':<8}{ 'w1':<8}{ 'w2':<8}{ 'b'")
        print("-" * 105)

        for i in range(len(inputs)):
            x1, x2 = inputs[i]
            d = targets[i]

            # Calculate net input (net = w*x + b)
            net = w1 * x1 + w2 * x2 + b

            # Apply activation function (e = function(net))
            e = bipolar_step_function(net)

            # Calculate error (error = d - e)
            error = d - e

            # Store delta values for printing (0 if no update)
            delta_w1_print = 0.0
            delta_w2_print = 0.0
            delta_b_print = 0.0

            # Update weights and bias if there's an error
            if error != 0:
                all_correct_in_epoch = False
                # Calculate delta changes
                delta_w1 = learning_rate * error * x1
                delta_w2 = learning_rate * error * x2
                delta_b = learning_rate * error

                # Update weights and bias
                w1 += delta_w1
                w2 += delta_w2
                b += delta_b

                delta_w1_print = delta_w1
                delta_w2_print = delta_w2
                delta_b_print = delta_b

            # Print current state (columns requested by user)
            print(f"{'x1':<5}{ 'x2':<5}{ 'd':<5}{ 'net':<8.2f}{ 'e':<8}{ 'error':<8}{ 'delta_w1_print':<8.2f}{ 'delta_w2_print':<8.2f}{ 'delta_b_print':<8.2f}")
            print("-" * 105)

        # If all patterns were correctly classified in this epoch, stop training
        if all_correct_in_epoch:
            break

    print(f"\nTraining finished for {gate_name} Gate in {epochs} epochs.")
    print(f"Final weights: w1={w1:.2f}, w2={w2:.2f}, bias={b:.2f}")

```

```

print("-----\n")
return w1, w2, b

# --- Bipolar AND Gate Definition ---
# Inputs (x1, x2) and Desired Outputs (d)
and_inputs = np.array([
    [-1, -1],
    [-1,  1],
    [ 1, -1],
    [ 1,  1]
])
and_targets = np.array([-1, -1, -1,  1])

# --- Bipolar OR Gate Definition ---
or_inputs = np.array([
    [-1, -1],
    [-1,  1],
    [ 1, -1],
    [ 1,  1]
])
or_targets = np.array([-1,  1,  1,  1])

# Train and print results for AND gate
train_perceptron(and_inputs, and_targets, "AND")

# Train and print results for OR gate
train_perceptron(or_inputs, or_targets, "OR")

```

--- Perceptron for AND Gate (Bipolar) ---
Initial weights: w1=0.0, w2=0.0, bias=0.0
Learning Rate: 1.0

Epoch 1:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
-1	-1	-1	0.00	0	-1	1.00	1.00	-1.00	1.00	1.00	-1.00
-1	1	-1	-1.00	-1	0	0.00	0.00	0.00	1.00	1.00	-1.00
1	-1	-1	-1.00	-1	0	0.00	0.00	0.00	1.00	1.00	-1.00
1	1	1	1.00	1	0	0.00	0.00	0.00	1.00	1.00	-1.00

Epoch 2:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
-1	-1	-1	-3.00	-1	0	0.00	0.00	0.00	1.00	1.00	-1.00
-1	1	-1	-1.00	-1	0	0.00	0.00	0.00	1.00	1.00	-1.00
1	-1	-1	-1.00	-1	0	0.00	0.00	0.00	1.00	1.00	-1.00
1	1	1	1.00	1	0	0.00	0.00	0.00	1.00	1.00	-1.00

Training finished for AND Gate in 2 epochs.
Final weights: w1=1.00, w2=1.00, bias=-1.00

--- Perceptron for OR Gate (Bipolar) ---
Initial weights: w1=0.0, w2=0.0, bias=0.0
Learning Rate: 1.0

Epoch 1:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
-1	-1	-1	0.00	0	-1	1.00	1.00	-1.00	1.00	1.00	-1.00
-1	1	1	-1.00	-1	2	-2.00	2.00	2.00	-1.00	3.00	1.00
1	-1	1	-3.00	-1	2	2.00	-2.00	2.00	1.00	1.00	3.00
1	1	1	5.00	1	0	0.00	0.00	0.00	1.00	1.00	3.00

Epoch 2:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
-1	-1	-1	1.00	1	-2	2.00	2.00	-2.00	3.00	3.00	1.00
-1	1	1	1.00	1	0	0.00	0.00	0.00	3.00	3.00	1.00
1	-1	1	1.00	1	0	0.00	0.00	0.00	3.00	3.00	1.00
1	1	1	7.00	1	0	0.00	0.00	0.00	3.00	3.00	1.00

Epoch 3:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
-1	-1	-1	-5.00	-1	0	0.00	0.00	0.00	3.00	3.00	1.00
-1	1	1	1.00	1	0	0.00	0.00	0.00	3.00	3.00	1.00
1	-1	1	1.00	1	0	0.00	0.00	0.00	3.00	3.00	1.00
1	1	1	7.00	1	0	0.00	0.00	0.00	3.00	3.00	1.00

Training finished for OR Gate in 3 epochs.
 Final weights: w1=3.00, w2=3.00, bias=1.00

```
import numpy as np

def unipolar_step_function(net):
    """Unipolar activation function."""
    return 1 if net >= 0 else 0

def train_perceptron(inputs, targets, gate_name, activation_function):
    """Trains a single-layer perceptron for a given gate using unipolar inputs and targets."""
    # Initialize weights and bias as specified
    w1, w2, b = 0.0, 0.0, 0.0
    learning_rate = 1.0 # Learning rate as specified
    epochs = 0

    print(f"--- Perceptron for {gate_name} Gate (Unipolar) ---")
    print(f"Initial weights: w1={w1}, w2={w2}, bias={b}")
    print(f"Learning Rate: {learning_rate}\n")

    # Training loop
    while True:
        epochs += 1
        print(f"Epoch {epochs}:")
        all_correct_in_epoch = True

        # Print header for the detailed table
        print(f"{'x1':<5}{'x2':<5}{'d':<5}{'net':<8}{'theta':<8}{'error':<8}{'dw1':<8}{'dw2':<8}{'db':<8}{'w1':<8}{'w2':<8}{'b':<8}")
        print("-" * 105)

        for i in range(len(inputs)):
            x1, x2 = inputs[i]
            d = targets[i]

            # Calculate net input (net = w*x + b)
            net = w1 * x1 + w2 * x2 + b

            # Apply activation function (e = function(net))
            e = activation_function(net)

            # Calculate error (error = d - e)
            error = d - e

            # Store delta values for printing (0 if no update)
            delta_w1_print = 0.0
            delta_w2_print = 0.0
            delta_b_print = 0.0

            # Update weights and bias if there's an error
            if error != 0:
                all_correct_in_epoch = False
                # Calculate delta changes
                delta_w1 = learning_rate * error * x1
                delta_w2 = learning_rate * error * x2
                delta_b = learning_rate * error

                # Update weights and bias
                w1 += delta_w1
                w2 += delta_w2
                b += delta_b

                delta_w1_print = delta_w1
                delta_w2_print = delta_w2
                delta_b_print = delta_b

            # Print current state (columns requested by user)
            print(f"{'x1':<5}{'x2':<5}{'d':<5}{'net':<8.2f}{'e':<8}{'error':<8}{'delta_w1_print':<8.2f}{'delta_w2_print':<8.2f}{'delta_b_print':<8.2f}")
            print("-" * 105)

        # If all patterns were correctly classified in this epoch, stop training
        if all_correct_in_epoch:
            break

    print(f"\nTraining finished for {gate_name} Gate in {epochs} epochs.")
    print(f"Final weights: w1={w1:.2f}, w2={w2:.2f}, bias={b:.2f}")
```

```

print("-----\n")
return w1, w2, b

# --- Unipolar AND Gate Definition ---
# Inputs (x1, x2) and Desired Outputs (d)
and_inputs_unipolar = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
and_targets_unipolar = np.array([0, 0, 0, 1])

# --- Unipolar OR Gate Definition ---
or_inputs_unipolar = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
or_targets_unipolar = np.array([0, 1, 1, 1])

# Train and print results for Unipolar AND gate
train_perceptron(and_inputs_unipolar, and_targets_unipolar, "AND", unipolar_step_function)

# Train and print results for Unipolar OR gate
train_perceptron(or_inputs_unipolar, or_targets_unipolar, "OR", unipolar_step_function)

```

--- Perceptron for AND Gate (Unipolar) ---
Initial weights: w1=0.0, w2=0.0, bias=0.0
Learning Rate: 1.0

Epoch 1:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
0	0	0	0.00	1	-1	-0.00	-0.00	-1.00	0.00	0.00	-1.00
0	1	0	-1.00	0	0	0.00	0.00	0.00	0.00	0.00	-1.00
1	0	0	-1.00	0	0	0.00	0.00	0.00	0.00	0.00	-1.00
1	1	1	-1.00	0	1	1.00	1.00	1.00	1.00	1.00	0.00

Epoch 2:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
0	0	0	0.00	1	-1	-0.00	-0.00	-1.00	1.00	1.00	-1.00
0	1	0	0.00	1	-1	-0.00	-1.00	-1.00	1.00	0.00	-2.00
1	0	0	-1.00	0	0	0.00	0.00	0.00	1.00	0.00	-2.00
1	1	1	-1.00	0	1	1.00	1.00	1.00	2.00	1.00	-1.00

Epoch 3:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
0	0	0	-1.00	0	0	0.00	0.00	0.00	2.00	1.00	-1.00
0	1	0	0.00	1	-1	-0.00	-1.00	-1.00	2.00	0.00	-2.00
1	0	0	0.00	1	-1	-1.00	-0.00	-1.00	1.00	0.00	-3.00
1	1	1	-2.00	0	1	1.00	1.00	1.00	2.00	1.00	-2.00

Epoch 4:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
0	0	0	-2.00	0	0	0.00	0.00	0.00	2.00	1.00	-2.00
0	1	0	-1.00	0	0	0.00	0.00	0.00	2.00	1.00	-2.00
1	0	0	0.00	1	-1	-1.00	-0.00	-1.00	1.00	1.00	-3.00
1	1	1	-1.00	0	1	1.00	1.00	1.00	2.00	2.00	-2.00

Epoch 5:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
0	0	0	-2.00	0	0	0.00	0.00	0.00	2.00	2.00	-2.00
0	1	0	0.00	1	-1	-0.00	-1.00	-1.00	2.00	1.00	-3.00
1	0	0	-1.00	0	0	0.00	0.00	0.00	2.00	1.00	-3.00
1	1	1	0.00	1	0	0.00	0.00	0.00	2.00	1.00	-3.00

Epoch 6:

x1	x2	d	net	theta	error	dw1	dw2	db	w1	w2	b
0	0	0	-3.00	0	0	0.00	0.00	0.00	2.00	1.00	-3.00
0	1	0	-2.00	0	0	0.00	0.00	0.00	2.00	1.00	-3.00
1	0	0	-1.00	0	0	0.00	0.00	0.00	2.00	1.00	-3.00
1	1	1	0.00	1	0	0.00	0.00	0.00	2.00	1.00	-3.00

Training finished for AND Gate in 6 epochs.

```
Final weights: w1=2.00, w2=1.00, bias=-3.00
```

```
-----
```

```
--- Perceptron for OR Gate (Unipolar) ---
```